

www.apnacollege.in | To exit full screen, press Esc

Functions in JS



JS



01:02 / 08:56



Functions in JS

Function Definition (telling JS)

```
function funcName( ) {  
    //do something  
}
```

```
function hello() {  
    console.log("hello");  
}
```

Function Calling (Using the function)

```
funcName( );
```

```
hello();
```



<> index.html

JS app.js

JS app.js > ...

```
1  function hello() {  
2      console.log("hello");  
3  }  
4  
5  hello();  
6  hello();  
7  hello();  
8  hello();  
9  hello();  
10 hello();
```



<> index.html

JS app.js

JS app.js > ...

```
1  function hello() {
2      console.log("hello");
3  }
4
5  function printName() {
6      console.log("apna college");
7      console.log("shradha khapra");
8  }
9
10 function print1to5() {
11     for(let i=1; i<=5; i++) {
12         console.log(i);
13     }
14 }
15
16 |
17
18
19 print1to5();
20
21 // printName();
```



<> index.html

JS app.js X

JS app.js > printPoem

```
1 function printPoem() {  
2   console.log("Twinkle Twinkle, little star");  
3   console.log("how I wonder what you are");  
4 }  
5  
6  
7 printPoem();
```



Functions with Arguments

Values we pass to the function

```
function funcName(arg1, arg2.. ) {  
    //do something  
}
```



<> index.html

JS app.js X

JS app.js > ...

```
1 function printInfo(name, age) {  
2   console.log(`${name}'s age is ${age}.`);  
3 }  
4  
5 printInfo("shradha", 23);
```

armankhan082020@gmail.com



Practice Qs

Create a Function that gives us the average of 3 numbers.

↓
a, b, c

$$\text{avg} = \frac{(a+b+c)}{3}$$



<> index.html

JS app.js

JS app.js > ...

```
1  ✓ function sum(a, b) {  
2    |   console.log(a+b);  
3    | }  
4  
5  ✓ function calcAvg(a, b, c) {  
6    |   let avg = (a+b+c)/3;  
7    |   console.log(avg);  
8    | }  
9
```

```
calcAvg(a: any, b: any, c: any): void
```

```
10 calcAvg(2, 4, 6)
```



JS x +

File | /Users/shradhakhapra/WebClassroom/JavaScript/index.html

Elements Console Sources Network >>

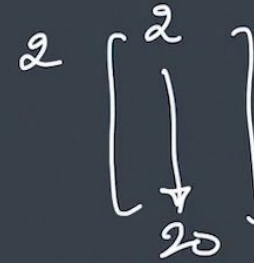
top Filter Default levels No Issues

```
4 app.js:7
> calcAvg(3, 3, 3);
3 app.js:7
< undefined
> calcAvg
< f calcAvg(a, b, c) {
    let avg = (a+b+c)/3;
    console.log(avg);
}
> Math.random
< f random() { [native code] }
> [1, 2, 3].push
< f push() { [native code] }
> [1, 2, 3].pop
< f pop() { [native code] }
> console.log
< f log() { [native code] }
>
```



Practice Qs

Create a Function that prints the multiplication table of a number.





<> index.html

JS app.js



JS app.js > printTable

```
1  function sum(a, b) {  
2    console.log(a+b);  
3  }  
4  
5  function calcAvg(a, b, c) {  
6    let avg = (a+b+c)/3;  
7    console.log(avg);  
8  }  
9  
10 function printTable(n) {  
11   for(let i=n; i<=n*10; i+=n) {  
12     console.log(i);  
13   }  
14 }  
15  
16  
17 calcAvg(2, 4, 6);
```



Return

return keyword is used to return some value from the function.



```
function funcName(arg1, arg2.. ) {  
  //do something ✓  
  return val;  
}
```



<> index.html

JS app.js

JS app.js > ...

```
1 function sum(a, b) {  
2   return a+b;  
3 }
```

```
4  
5 sum(sum(1, 2), 3);  
6 console.log();  
7
```

ankhn082020@gmail.com



<> index.html

JS app.js



JS app.js > sum

```
1  function sum(a, b) {  
2      console.log("hello");  
3      console.log("hello");  
4      return a+b;  
5      console.log("hello2");  
6      console.log("hello2");  
7  }  
8  
9  
10 console.log(sum(1, 2));  
11
```



<> index.html

JS app.js

JS app.js > isAdult

```
4     return a+b;
5     console.log("hello2");
6     console.log("hello2");
7 }
8
9 function isAdult(age) {
10     if(age >= 18) {
11         return "adult";
12     } else {
13         return "not adult";
14     }
15 }
16
17
18
```



Practice Qs

Create a Function that returns the sum of numbers from 1 to n.

1 to n (argument)
sum



Practice Qs

Create a Function that returns the sum of numbers from 1 to n.

```
let sum = 0;  
for (let i = 1; i <= n; i++) {  
    sum = sum + i; // sum  
}
```

$n = 7$

1
2
3
4
5
6
7



<> index.html

JS app.js

JS app.js >  getSum

```
1  function getSum(n) {  
2      let sum = 0;  
3  
4      for(let i=1; i<=n; i++) {  
5          sum += i;  
6      }  
7  
8      return sum;  
9  }
```



<> index.html

JS app.js



JS app.js > ...

```
1  let str = ["hi", "hello", "bye", "!"];
2
3  function concat(str) {
4      let result;
5
6      for(let i=0; i<str.length; i++) {
7          result += str[i];
8      }
9
10     return result;
11 }
```

armankhan082020@gmail.com



<> index.html

JS app.js

JS app.js >  concat

```
1  let str = ["hi", "hello", "bye", "!"];
2
3  function concat(str) {
4      let result = "";
5
6      for(let i=0; i<str.length; i++) {
7          result += str[i];
8      }
9
10     return result;
11 }
```

armankhan082020@gmail.com



JS x +

File | /Users/shradhakhapra/WebClassroom/JavaScript/index.html

Elements Console Sources Network >>

top Filter Default levels No Issues

```
> str
< ▶ (4) ['hi', 'hello', 'bye', '!']

> concat
< f concat(str) {
    let result = "";

    for(let i=0; i<str.length; i++) {
        result += str[i];
    }

    return result;
}

> concat(str);
< 'hihellobye!'

> |
```



Scope

Scope determines the **accessibility** of variables, objects, and functions from different parts of the code.

- Function
- Block
- Lexical



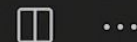
Function Scope

Variables defined inside a function are not accessible (visible) from outside the function.



<> index.html

JS app.js



JS app.js >  calSum

```
1  let sum = 54; //Global Scope
2
3  function calSum(a, b) {
4      let sum = a+b; //Function Scope
5      console.log(sum);
6  }
7
8  calSum(1, 2);
9
10
```



Block Scope

Variables declared inside a { } block cannot be accessed from outside the block.





<> index.html

JS app.js



JS app.js > ...

```
1  {  
2    |   var a = 25;  
3  }  
4  
5  console.log(a);
```



<> index.html

JS app.js



JS app.js > ...

```
1 {  
2   let a = 25;  
3 }  
4  
5 console.log(a);
```





<> index.html

JS app.js



JS app.js > ...

```
9    // }
10
11    // console.log(i);
12
13    let age = 25;
14    if(age >= 18) {
15        let str = "adult";
16    }
17
18    console.log(str);
```



Lexical Scope → *nested function*

a variable defined outside a function can be accessible inside another function defined after the variable declaration.

The opposite is NOT true.





<> index.html

JS app.js



JS app.js > outerFunc

```
1  function outerFunc() {  
2      let x = 5;  
3      let y = 6;  
4      function innerFunc() {  
5          console.log(x);  
6      }  
7      innerFunc();  
8  }  
9  }
```





<> index.html

JS app.js



JS app.js > ...

```
1  function outerFunc() {  
2      function innerFunc() {  
3          console.log(x);  
4          console.log(y);  
5      }  
6      let x = 5;  
7      let y = 6;  
8  
9  
10     innerFunc();  
11 }  
12  
13 //hoisting
```



JS x +

File | /Users/shradhakhapra/WebClassroom/JavaScript/index.html

Elements Console Sources Network >> 1

top Filter Default levels No Issues

```
> outerFunc();
```

✖ ▶ Uncaught ReferenceError: a is not defined app.js:9

```
    at outerFunc (app.js:9:17)
    at <anonymous>:1:1
```

```
> |
```



JS x +

File | /Users/shradhakhapra/WebClassroom/JavaScript/index.html

Elements Console Sources Network >> 2

top Filter Default levels No Issues

```
> outerFunc();
```

Uncaught ReferenceError: a is not defined app.js:9
at outerFunc (app.js:9:17)
at <anonymous>:1:1

```
> innerFunc();
```

Uncaught ReferenceError: innerFunc is not defined VM1096:1
at <anonymous>:1:1

```
>
```



<> index.html

JS app.js X

JS app.js > outerFunc

```
1  function outerFunc() {  
2      let x = 5;  
3      let y = 6;  
4      function innerFunc() {  
5          let a = 10;  
6          console.log(x);  
7          console.log(y);  
8      }  
9      console.log(a);  
10     innerFunc();  
11 }
```

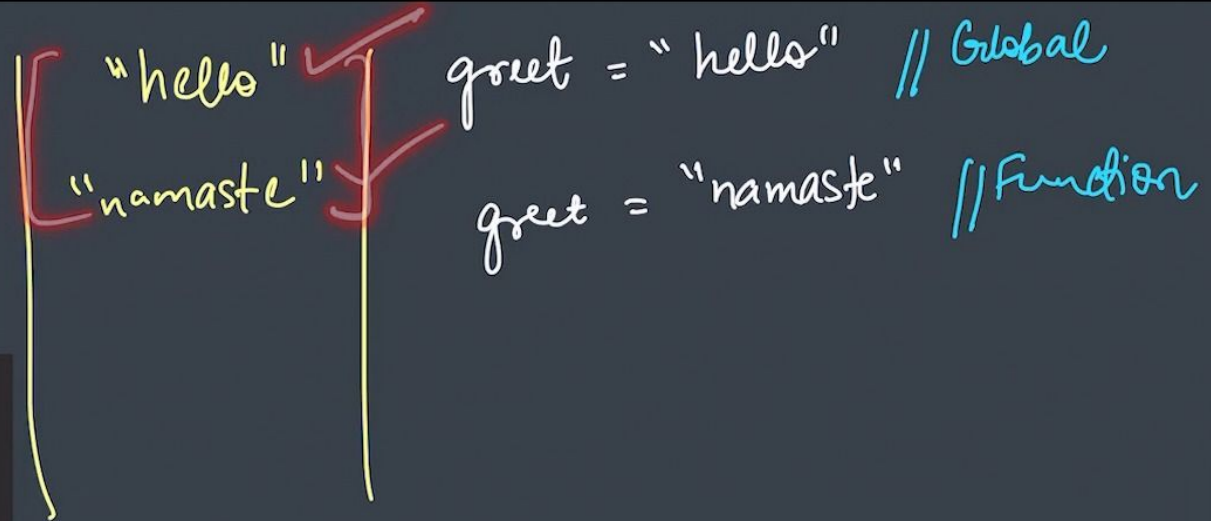
armankhan082020@gmail.com



Practice Qs

What will be the output?

```
let greet = "hello";  
  
function changeGreet() {  
  let greet = "namaste";  
  console.log(greet);  
  function innerGreet() {  
    console.log(greet);  
  }  
}  
  
console.log(greet);  
changeGreet();
```


"hello"
"namaste"
greet = "hello" // Global
greet = "namaste" // Function

armankhan082020@gmail.com





<> index.html

JS app.js



JS app.js > ...

```
1 let greet = "hello"; |
2
3 function changeGreet() {
4     let greet = "namaste";
5     console.log(greet);
6
7     function innerGreet() {
8         console.log(greet); //lexical scope
9     }
10
11     innerGreet();
12 }
13
14 console.log(greet);
15 changeGreet();
```



Function Expressions

```
const variable = function(arg1, arg2..) {  
  //do or return something  
}
```

```
const sum = function(a, b) {  
  return a + b;  
}  
  
sum(2, 3);
```





<> index.html

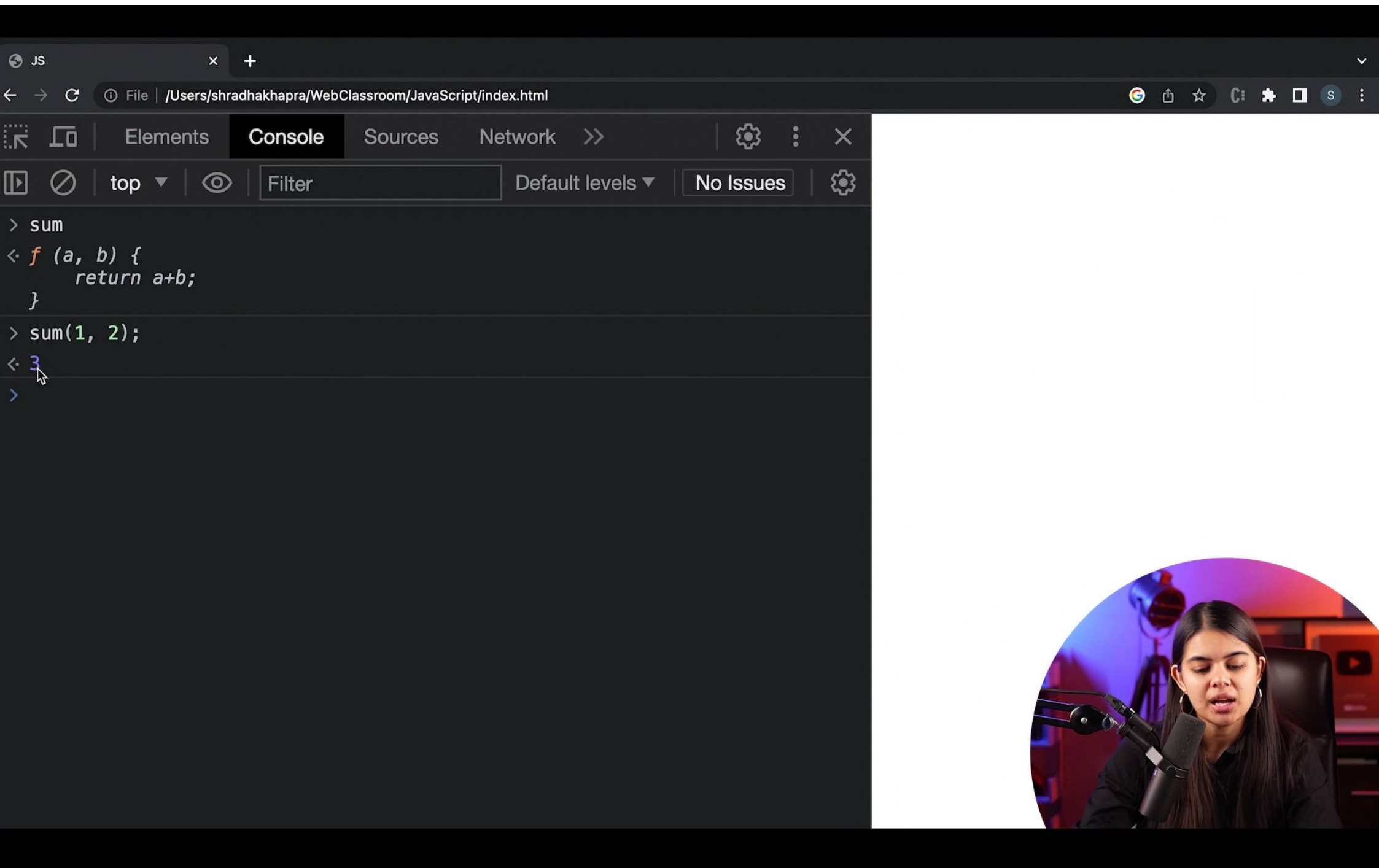
JS app.js



JS app.js > [e] sum

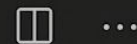
```
1 let name = "shradha";  
2  
3 let sum = function(a, b) {  
4   ...return a+b;  
5 }
```





<> index.html

JS app.js



JS app.js > [x] sum

```
1 let name = "shradha";  
2  
3 let sum = function(a, b) {  
4   return a+b;  
5 }  
6  
7 let hello = function() {  
8   console.log("hello");  
9 }
```

aiman.khan082020@gmail.com





<> index.html

JS app.js



JS app.js >  hello

```
1 let name = "shradha";
2 let x = 5;
3
4 let sum = function(a, b) {
5   return a+b;
6 }
7
8 let hello = function() {
9   console.log("hello");
10 }
11
12 hello = function() {
13   console.log("namaste");
14 }
```

armankhan082020@gmail.com



JS x +

File | /Users/shradhakhapra/WebClassroom/JavaScript/index.html

Elements Console Sources Network >> | ⚙️ ⋮ ✕

top ▾ | 🔍 Filter | Default levels ▾ | No Issues | ⚙️

```
> hello
< f () {
  console.log("namaste");
}
> hello();
namaste app.js:13
< undefined
>
armankhan082020@gmail.com
```



Higher Order Functions

A function that does one or both :

- takes one or multiple functions as arguments
- returns a function



Higher Order Functions

Takes one or multiple functions as arguments

```
function multipleGreet(func, n) {  
  for(let i=1; i<=n; i++) {  
    func();  
  }  
}  
  
let greet = function() {  
  console.log("hello");  
}  
  
multipleGreet(greet, 2);
```

2 hello



<> index.html

JS app.js



www.apnacollege.in – To exit full screen, press Esc



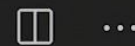
JS app.js > multipleGreet

```
1 function multipleGreet(func, count) { //higher order function
2     for(let i=1; i<=count; i++) {
3         func();
4     }
5 }
6
7 let greet = function() {
8     console.log("hello");
9 }
10
11 multipleGreet(greet, 1000);
12
```



<> index.html

JS app.js X



JS app.js > [🔍] greet

```
1  function multipleGreet(func, count) { //higher order function
2      for(let i=1; i<=count; i++) {
3          func();
4      }
5  }
6
7  let greet = function() {
8      console.log("hello");
9  }
10
11  multipleGreet(function() {console.log("namaste")}, 1000);
12
```



Higher Order Functions

Returns a function

```
function oddEvenTest(request) {  
  if(request == "odd") {  
    return function(n) {  
      console.log(!(n%2 == 0));  
    }  
  } else if(request == "even") {  
    return function(n) {  
      console.log(n%2 == 0);  
    }  
  } else {  
    console.log("wrong request");  
  }  
}
```





<> index.html

JS app.js



JS app.js > [🔍] request

```
1  function oddOrEvenFactory(request) {
2      if(request == "odd") {
3          let odd = function(n) {
4              console.log(!(n%2 == 0));
5          }
6
7          return odd;
8      } else if(request == "even") {
9          let even = function(n) {
10             console.log(n%2 == 0);
11         }
12
13         return even;
14     } else {
15         console.log("wrong request");
16     }
17 }
18
19 let request = "odd"; //even
```



JS x +

File | /Users/shradhakhapra/WebClassroom/JavaScript/index.html

Elements Console Sources Network >> | ⚙️ ⋮ ✕

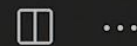
⏮ ⏪ top ▼ 👁 Filter Default levels ▼ No Issues ⚙️

```
> request = "even";  
< 'even'  
> request  
< 'even'  
> let func = oddOrEvenFactory(request);  
< undefined  
> func  
< f (n) {  
    console.log(n%2 == 0);  
}  
> func(3);  
false app.js:10  
< undefined  
> request = "even";|
```



<> index.html

JS app.js X



JS app.js > ...

```
1 function oddOrEvenFactory(request) {  
2   if (request === "odd") {  
3     return function(n) {  
4       console.log(!(n%2 === 0));  
5     }  
6   } else if (request === "even") {  
7     return function(n) {  
8       console.log(n%2 === 0);  
9     }  
10  } else {  
11    console.log("wrong request");  
12  }  
13 }  
14  
15 let request = "odd"; //even
```



Methods

Actions that can be performed on an object.

```
const calculator = {  
  add: function(a, b) {  
    return a + b;  
  },  
  sub: function(a, b) {  
    return a - b;  
  },  
  mul: function(a, b) {  
    return a * b;  
  }  
};
```



JS x +

File | /Users/shradhakhapra/WebClassroom/JavaScript/index.html

Elements Console Sources Network >>

top Filter Default levels No Issues

```
> calculator
< {num: 55, add: f, sub: f, mul: f} i
  ▶ add: f (a, b)
  ▶ mul: f (a, b)
    num: 55
  ▶ sub: f (a, b)
  ▶ [[Prototype]]: Object

> calculator.num
< 55

> calculator.add
< f (a, b) {
    return a+b;
}

> calculator.add(1, 2);
< 3
>
```



Methods

Actions that can be performed on an object.

```
const calculator = {  
  add: function(a, b) {  
    return a + b;  
  },  
  sub: function(a, b) {  
    return a - b;  
  },  
  mul: function(a, b) {  
    return a * b;  
  }  
};
```

{ arrays
strings } methods
↓
objects



Methods (Shorthand)

func's

```
const calculator = {  
  add(a, b) {  
    return a + b;  
  },  
  sub(a, b) {  
    return a - b;  
  }  
};
```



JS (Part 6)

Practice Questions

[Note - Some questions might take more time to solve as it will take time for any coding beginner to build logic for approaching such programming problems.

These questions are based on programming logic, not on development. So, please don't hesitate to look at the solution and try to understand the approach used.]

Qs1. Write a JavaScript function that returns array elements larger than a number.

Qs2. Write a JavaScript function to extract unique characters from a string.

Example: str = "abcdabdefgggh"

ans = "abcdefgh"

Qs3. Write a JavaScript function that accepts a list of country names as input and returns the longest country name as output.

Example : country = ["Australia", "Germany", "United States of America"]

output : "United States of America"

Qs4. Write a JavaScript function to count the number of vowels in a String argument.

Qs5. Write a JavaScript function to generate a random number within a range (start, end).